

Analysis: Cheating a Boolean Tree

This is an easy exercise in dynamic programming.

Let us define for each node v in the tree, $F(v, x)$ to be the smallest number of gates we need to flip in order to make the output of v be x (0 or 1). The value $F(v, x)$ can be computed using dynamic programming as follows.

If v is a leaf with input value 0, then $F(v, 0) = 0$ -- no gate needs to be changed; and $F(v, 1)$ can be assigned to -1 or some really big value to indicate "mission impossible".

If v has two children, u and w , and the gate at v is OR, then $F(v, 0)$ can be computed by taking the better of the following options.

1. Do not change the gate and use the plan for $F(u, 0)$ and $F(w, 0)$;
2. Change the gate to AND and use the plan for $F(u, 0)$;
3. Change the gate to AND and use the plan for $F(w, 0)$.

So

$$F(v, 0) = \min\{ F(u, 0) + F(w, 0), 1 + F(u, 0), 1 + F(w, 0) \}.$$

The other cases are similar. One can compute the F values iteratively bottom-up or recursively top-down.

In addition to the solution above, we introduce some interesting observations. Look closely at the formula above. Suppose we want the output 0 at the top, when you start a top-down computation, you will only use values $F(_, 0)$ and never use any $F(_, 1)$. Similarly, if the desired output at the top is 1, you will never need to care about the values for $F(_, 0)$. Furthermore, in computing $F(v, 1)$, you never want to change an OR gate to an AND gate. The deep reason for these is that, when there is no negation gate involved, the circuit computes a monotone function, and you will never want to change an output from 1 to 0 in the middle.

By de Morgan's law, if we interchange all the input values, change all the gates to the opposite type, and change the desired output from 0 to 1 (or 1 to 0), we obtain a dual problem, and the minimum number of gates one needs to change remains the same. So we can assume that the desired value is 1 (or 0, depends on your taste), and forget about implementing half of the cases.

Analysis: PermRLE

Section A. The Hamiltonian cycle in a small world

A Hamilton cycle in a graph is a cycle that visits each node exactly once. Given a weighted, directed, complete graph on n nodes, there are $(n-1)!$ distinct Hamiltonian cycles. It is well known that the problem of finding the shortest (or longest) Hamilton cycle is NP-hard. It is also known to many contestants that, for n as small as 20, dynamic programming makes a difference of $n \cdot 2^n$ vs. $n!$, which is the difference between a second and an eternity.

Let's have a look at the $n \cdot 2^n$ DP trick, in case you have not seen it before.

Without loss of generality, we may view node 0 as the start point of the cycle, as well as its end point. For any subset A of the node set V and any node x in A , we define

$dp[A][x] :=$ The shortest path that starts from x , visits each point in A exactly once and ends up at node 0. (*)

To clarify, 0 does not necessarily belong to A , but we do count the length of the edge from the last point to node 0. Thus the problem of finding the shortest Hamilton cycle is just $dp[V][0]$. (Convince yourself, maybe looking at (*).)

We need to compute $dp[A][x]$. For the easy cases where $A = \{x\}$, the answer is just the length of edge $x \rightarrow 0$. Otherwise, we focus on the first step of the path. If the first step is $x \rightarrow y$, with edge length q , then we pay $dp[A - x][y] + q$. In general, $dp[A][x]$ is

- $\text{length}(x \rightarrow 0)$, if $A = \{x\}$.
- $\min \{ dp[A - x][y] + \text{length}(x \rightarrow y) \mid y \in A - x \}$, if $|A| > 1$.

Section B. Wrap everything into a small world

For any string, define the *number of switches* to be the number of times adjacent characters are different in the string. We want to find a permutation that transforms $\mathbf{S}$ to one $\mathbf{S}'$ where the number of switches is minimal. Assume the length of $\mathbf{S}$ is $\mathbf{mk}$. Then $\mathbf{S}$ can be viewed as a string with $\mathbf{m}$ blocks of length $\mathbf{k}$.

Now we introduce a visual aid to simplify our writing. Let us draw the string $\mathbf{S}$ as $\mathbf{m}$ rows, each block on a single row. The key image is to count the number of switches one *column* at a time.

Let us take a semi-concrete example. Suppose that at one point we have decided that 5 is permuted to the 7th position, and that 2 goes to the 8th position. Then without knowing the rest of the permutation, we can inspect the 5th and 2nd characters in each block. Suppose that in Z of the blocks the 5th and the 2nd characters are different, then we know that in any such permutation, we will have to pay the price of Z .

The one exception is the last element of the permutation. In all cases but one, we simply wrap around to the beginning because the end of each k -block touches the beginning of the next k -block in the string, except for the last character in the string. We can handle both cases if we fix the last element of the permutation by trying all possibilities.

Next, we reduce our problem to the one in Section A. Suppose we fix $\mathbf{T}$ as the last element in the permutation. Define a weighted, directed, complete graph G on $\mathbf{k}$ vertices $\{1, 2, \dots, \mathbf{k}\}$. The

weight on the edge $x \rightarrow y$ is

- the number of blocks where the x -th character is different from the y -th character in the same block. (if $x \neq T$)
- the number of blocks (excluding the last one) where the x -th character is different from the y -th character in the *next* block. (if $x = T$)

It is easy to check that for any permutation, the number of switches is the same as the length of the corresponding Hamiltonian cycle in G .

We have k different choices for T . For each T , finding the shortest Hamilton cycle takes $O(2^k k)$ time. The construction of the graph takes $O(k^2 m) = O(k |S|)$ time for each T ; it is also easy to construct in $O(k^2 m)$ time the graphs for all the T 's. The running time of the solution is $O(2^k k^2 + k |S|)$.

Analysis: Star Wars

This problem offers a nice excursion into some basic pictures in algebra and geometry, and certainly some programming basics.

The problem is to find the smallest power Y_0 , such that there is a position where power Y_0 is good enough to reach all of the ships. Clearly, any power bigger than Y_0 is big enough, while any power smaller than Y_0 is not. So, the first step towards the solution is to use the binary search. Reduce the problem of finding the smallest Y to a sequence of easier problems of deciding whether a given Y is big enough. Below we can focus on the decision problem for a given Y instead of the original optimization problem.

For a given Y , we have a requirement that each ship i satisfies

$$(1) \quad (|x_i - x| + |y_i - y| + |z_i - z|) \leq p_i Y$$

Geometrically, this means that the point (x, y, z) for the cruiser must be in the octahedron centered at (x_i, y_i, z_i) . Each of the N ships gives one octahedron, and a good position for the cruiser exists if and only if all these N octahedra intersect.

Algebraically, (1) is equivalent to the following set of inequalities (prove it!)

$$\begin{aligned} x + y + z &\leq x_i + y_i + z_i + p_i Y \\ x + y + z &\geq x_i + y_i + z_i - p_i Y \\ x + y - z &\leq x_i + y_i - z_i + p_i Y \\ x + y - z &\geq x_i + y_i - z_i - p_i Y \\ x - y + z &\leq x_i - y_i + z_i + p_i Y \\ x - y + z &\geq x_i - y_i + z_i - p_i Y \\ -x + y + z &\leq -x_i + y_i + z_i + p_i Y \\ -x + y + z &\geq -x_i + y_i + z_i - p_i Y \end{aligned}$$

For the geometrically inclined, each octahedron is associated with one of the four directions given by the vectors $(1, 1, 1)$, $(1, 1, -1)$, $(1, -1, 1)$ and $(-1, 1, 1)$. Each pair of inequalities states that the projection (the inner product) of (x, y, z) on a given direction vector must be in a certain range.

Now we have the problem of solving a set of inequalities of the form

$$\begin{aligned} A &\leq x + y + z \leq B \\ C &\leq x + y - z \leq D \\ E &\leq x - y + z \leq F \\ G &\leq -x + y + z \leq H \end{aligned}$$

where A, B, C, D, E, F, G and H are given. In general, this is a linear program. But it is such a trivial one that we do not need to pull out any serious linear programming algorithms.

Certainly, for the solution to exist, we must have $A \leq B$, $C \leq D$, $E \leq F$, and $G \leq H$. But these conditions are not enough. The inequalities can be rewritten as

$$\begin{aligned} A - x &\leq y + z \leq B - x \\ G + x &\leq y + z \leq H + x \end{aligned}$$

$$\begin{aligned} C - x &\leq y - z \leq D - x \\ -F + x &\leq y - z \leq -E + x \end{aligned}$$

As long as $y + z$ and $y - z$ have solutions, we can get y and z . We want to see whether there is an x such that the range $[A - x, B - x]$ intersects $[G + x, H + x]$, and the range $[C - x, D - x]$ intersects $[-F + x, -E + x]$.

It is easy to see that in order for the first two ranges to intersect, we must have

$$(2) \quad x \text{ in } [(A - H) / 2, (B - G) / 2].$$

And for the other two ranges, we must have

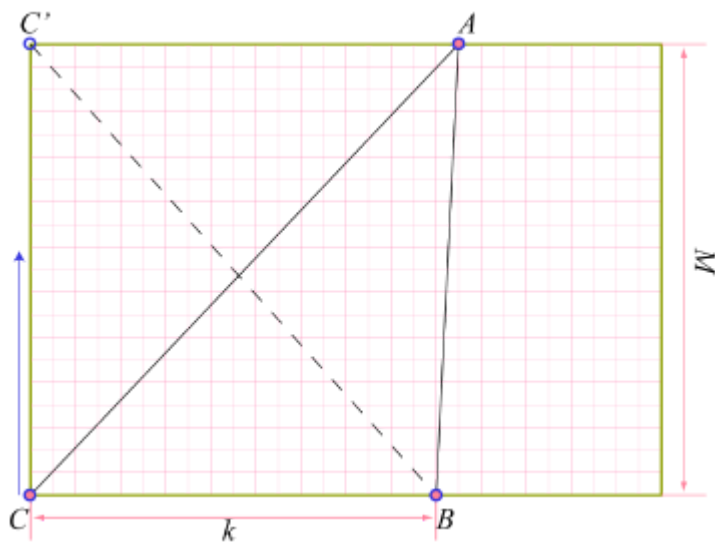
$$(3) \quad x \text{ in } [(C + E) / 2, (D + F) / 2].$$

The last step of our solution is simply to decide whether the two intervals in (2) and (3) have a non-empty intersection.

Analysis: Triangle Areas

This problem was originally in the form of a little puzzle: For how many numbers is the area possible? The answer is that there are **MN** such numbers, for any integer $0 < \mathbf{A} \leq \mathbf{MN}$, we can find a triangle with area $\mathbf{A}/2$ formed by integer points on the graph paper.

The following picture gives the key step in the proof, as well as a solution to our problem.



Assume the integer part of $\mathbf{A}/\mathbf{M}$ is k , we have $kM \leq \mathbf{A} \leq (k+1)M$. Denote $S(XYZ)$ the area of triangle $\triangle XYZ$. Clearly $2S(ABC) = kM$ and $2S(ABC') = (k+1)M$. Now take a point C^* from your pocket, put it on C , and move it up towards C' , one unit at a time. What can we say about the quantity $2S(ABC^*)$?

- It starts with kM and ends with $(k+1)M$.
- It is monotone increasing because the distance from C^* to AB is monotone.
- It is always an integer.
- The journey takes exactly $\mathbf{M}$ steps.

Putting these together, the simple conclusion is that $2S(ABC^*)$ will hit every integer between kM and $(k+1)M$, including $\mathbf{A}$.

Exercises

(1) For the careful readers, there is one more thing we did not prove yet. There is no way to form a triangle on the graph paper with an area bigger than $\mathbf{MN}/2$. Find a simple reason for this.

(2) We argued that $2S(ABC^*)$ must hit $\mathbf{A}$ because it will hit every integer between kM and $(k+1)M$. Reason directly that, while C^* is moving upward, $S(ABC^*)$ will increase by 0.5 every time C^* moves one unit higher.